

# ADAPTIVE MULTI-AGENT DATA SCIENCE COPILOT

## INPUTS CONSIDERED

### PROBLEM STATEMENT

To Design and evaluate data science language models that can adapt to different data science models

### PROBLEM UNDERSTANDING

- Industries work with large datasets, but creating ML models manually takes time.
- Users need to clean data, choose models, tune parameters, compare results, and explain predictions.
- This project reduces manual effort by automating the complete data science workflow.
- The system uses multiple AI agents, where each agent handles one responsibility.
- The raw dataset is processed by the ML engine, while AI agents mainly use dataset metadata for decision-making.
- This makes the workflow faster, organized, and more privacy-aware.

## INPUTS CONSIDERED

| Input Category       | Description                                                                         |
|----------------------|-------------------------------------------------------------------------------------|
| Structured Dataset   | CSV, Excel, JSON, or Parquet file uploaded by the user                              |
| Numerical Features   | Numeric columns used for analysis and model training                                |
| Categorical Features | Text, label, category, or Boolean columns used for encoding and prediction          |
| Time-Series Inputs   | Date or timestamp-based data used for forecasting                                   |
| Target Column        | Output column to be predicted, such as class, price, sales, churn, or fraud         |
| Missing Values       | Empty or null values detected in the dataset                                        |
| Duplicate Records    | Repeated rows identified before model training                                      |
| Dataset Metadata     | Row count, column count, data types, feature list, and suggested target column      |
| Model Results        | Best model, evaluation metrics, explainability results, and downloadable model file |

## KEY INNOVATION

- Uses multiple AI agents to automate the data science workflow.
- Detects the machine learning task automatically.
- Generates preprocessing and modeling steps based on dataset metadata.
- Trains and compares multiple models.
- Provides explainable AI output using SHAP.
- Produces a professional final report and downloadable model.

## TECHNOLOGIES USED

React + Next.js, FastAPI, CrewAI, Gemini API, XGBoost, Optuna, SHAP, ChromaDB, MLflow, Docker.

# PROCESS TO BE FOLLOWED

## SYSTEM WORKFLOW ARCHITECTURE

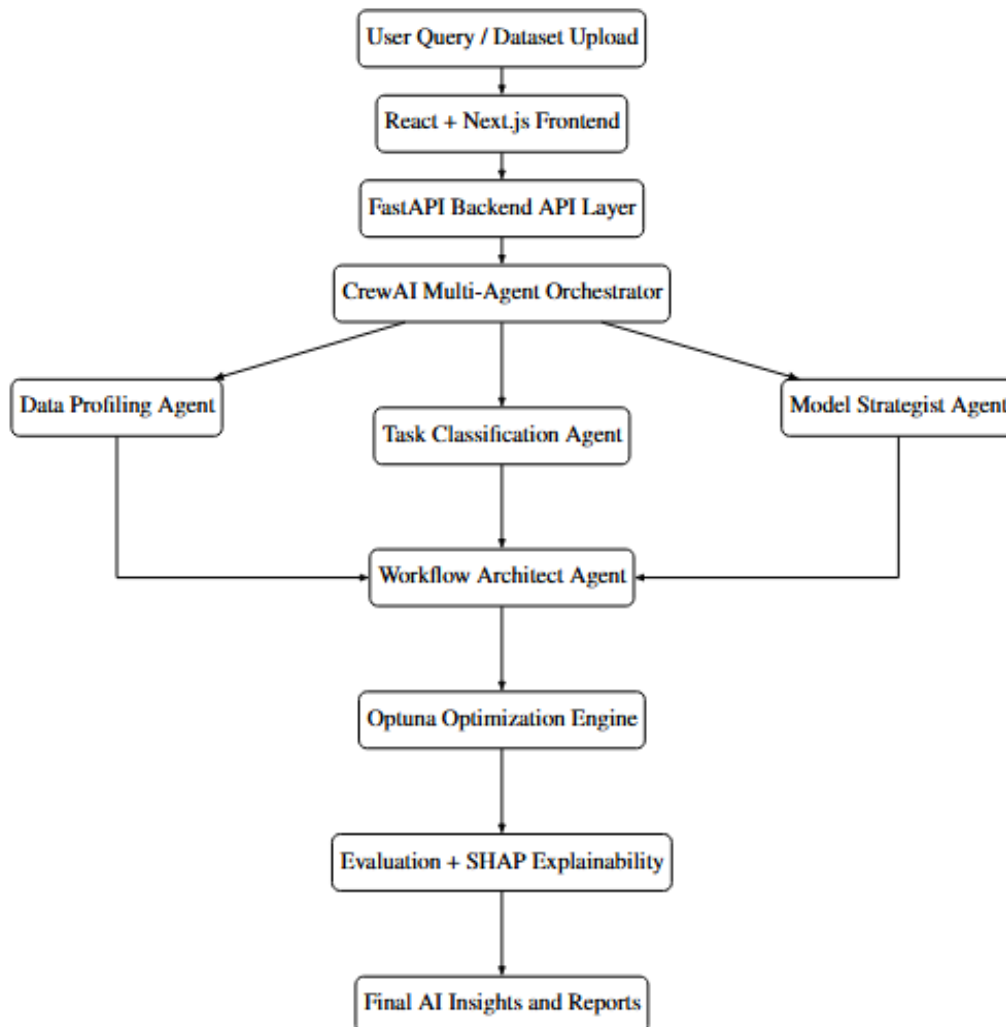


Figure 1: Adaptive Multi-Agent AI Workflow Architecture

## ARCHITECTURE EXPLANATION

### STEP 1: USER UPLOADS DATASET

- The user uploads a dataset through the web dashboard.
- The dataset may be in CSV, Excel, JSON, or Parquet format.
- The backend receives the file and stores it securely in the project data folder.
- A unique job ID is created for tracking the workflow.

### STEP 2: DATASET LOADING

- The FastAPI backend reads the uploaded file.
- pandas is used to load the dataset.
- The system checks the file type and loads it using the correct method.
- If the file type is unsupported, the system stops the process and returns an error.

### **STEP 3: DATA PROFILING**

- The dataset is analyzed using pandas and NumPy.
- The system extracts important metadata from the dataset.
- It identifies numerical, categorical, and datetime columns.
- It checks missing values and duplicate records.
- It suggests a possible target column.
- It detects whether the dataset contains time-series information.

### **STEP 4: METADATA CREATION**

- The extracted metadata is converted into a compact summary.
- This metadata includes row count, column count, feature types, missing percentage, duplicate count, target column, class count, and time-series details.
- AI agents use this metadata for reasoning.
- Raw user data is not directly sent to the AI agents.

### **STEP 5: DATA QUALITY ANALYSIS**

- The Data Agent reviews the metadata.
- It identifies data issues such as missing values, duplicates, imbalance, or unsuitable target columns.
- It gives a summary of the dataset condition.
- This helps the system decide what cleaning or preprocessing is needed.

### **STEP 6: TASK DETECTION**

- The Task Agent analyzes the metadata.
- It detects the type of machine learning problem.
- The task can be classification, regression, clustering, or forecasting.
- This step is important because the remaining workflow depends on the detected task.

### **STEP 7: WORKFLOW PLANNING**

- The Workflow Agent prepares the preprocessing plan.
- It decides how to handle missing values.
- It plans encoding for categorical columns.
- It plans scaling for numerical columns.
- It prepares the dataset for model training.

### **STEP 8: MODEL SELECTION**

- The Model Strategist Agent selects suitable models based on the detected task.
- Classification uses Logistic Regression, Random Forest, and XGBoost.
- Regression uses Linear Regression, Random Forest Regressor, and XGBoost Regressor.
- Clustering uses KMeans and DBSCAN.
- Forecasting uses ARIMA and Exponential Smoothing.

## **STEP 9: DATA PREPROCESSING**

- The ML engine applies preprocessing using scikit-learn.
- Missing values are handled.
- Categorical features are encoded.
- Numerical features are scaled when required.
- The target column is separated from input features.

## **STEP 10: MODEL TRAINING**

- Multiple models are trained based on the task type.
- scikit-learn, XGBoost, and stats models are used for training.
- Classification and regression models are trained using train-test split.
- Forecasting models are trained using time-series methods.
- Clustering models are trained without a target column.

## **STEP 11: HYPERPARAMETER OPTIMIZATION**

- Optuna is used to improve model performance.
- It tries different parameter combinations.
- The best parameter values are selected based on model performance.
- This improves accuracy, reliability, and robustness.

## **STEP 12: MODEL EVALUATION**

- The Evaluation Agent compares all trained models using task-specific metric and best model selected.
- Classification: Accuracy, Precision, Recall, F1-score, ROC-AUC.
- Regression: RMSE, MAE, R<sup>2</sup> Score.
- Clustering: Silhouette Score and number of clusters.
- Forecasting: AIC, BIC, and forecast values.

## **STEP 13: EXPLAINABILITY**

- SHAP is used to explain model predictions.
- The system identifies which features influenced the model output.
- The Explainability Agent converts technical explainability results into understandable insights.

## **STEP 14: FINAL REPORT AND MODEL EXPORT**

- The Report Agent generates the final AI report.
- The best-performing model is saved as a downloadable .pkl file.
- MLflow logs experiment details and model metrics.
- ChromaDB stores workflow memory for future reference.
- The dashboard displays the final output to the user.

## EXPECTED OUTPUT

### INTELLIGENT DASHBOARD OUTPUT

• The platform provides workflow monitoring, AI agent logs, benchmarking charts, explainability visualizations, and optimization history through an interactive dashboard.

### DATASET INTELLIGENCE OUTPUT

#### Uploaded Dataset Summary

- Total Rows: 50,000
- Numerical Features: 18
- Categorical Features: 6
- Missing Values: 3.2%
- Detected Task: Classification

### AI AGENT COLLABORATION OUTPUT

• AI agents automatically perform task detection, preprocessing pipeline generation, model selection, and Optuna-based optimization.

### MODEL BENCHMARKING OUTPUT

| Model               | Accuracy | F1-Score |
|---------------------|----------|----------|
| Logistic Regression | 88.4%    | 0.86     |
| Random Forest       | 93.7%    | 0.92     |
| XGBoost             | 96.2%    | 0.95     |

XGBoost with Optuna Optimization [Reason - Superior nonlinear learning and optimized performance.]

### EXPLAINABLE AI OUTPUT

• SHAP-based explainability generates feature importance analysis, prediction reasoning, and interpretable AI insights.

## FINAL EXPECTED OUTCOME

The proposed system delivers:

- adaptive workflow automation
- intelligent model optimization
- explainable AI insights
- scalable enterprise architecture
- autonomous multi-agent orchestration

The platform transforms traditional ML workflows into an adaptive AI-assisted analytics ecosystem for enterprise applications.

**OUTPUT LINKS: GITHUB:** <https://github.com/Udhayi2564/datascience-model.git>

**GOOGLE DRIVE:** <https://drive.google.com/drive/folders/170TNIDg8dtLWlQpeGz70ISsgM653vdJM?usp=sharing>